

Type: MCQ

Q1. What is the pseudocode to find the minimum element in the given array? (0.5)

1. ******
 $maxval \leftarrow A[0]$
for $i \leftarrow 1$ **to** $n - 1$ **do**
 if $A[i] < maxval$
 $maxval \leftarrow A[i]$
return $maxval$
2.
 $maxval \leftarrow A[0]$
for $i \leftarrow 1$ **to** $n - 1$ **do**
 if $A[i] > maxval$
 $maxval \leftarrow A[i]$
return $maxval$
3.
 $minval \leftarrow A[0]$
for $i \leftarrow 1$ **to** $n - 1$ **do**
 if $A[i] > minval$
 $minval \leftarrow A[i]$
return $minval$
4.
 $minval \leftarrow A[0]$
for $i \leftarrow 1$ **to** $n - 1$ **do**
 if $A[i] < minval$
 $minval \leftarrow A[i]$
return $maxval$

Q2. Consider the following pseudocode for the sequential search algorithm. (0.5)

```
 $i \leftarrow 0$   
while  $i < n$  and  $A[i] \neq K$  do  
    Missing statement  
if  $i < n$  return  $i$   
else return  $-1$ 
```

What is the missing statement in the above code snippet?

1. $\text{if } A[i] == K$
2. ****** $i = i + 1$
3. $i = i - 1$
4. No statement is required.

Q3. _____ is the solution of following recurrence relation.

$$T(n) = 2T(n-1) + 1 \quad \text{for } n > 0,$$

$T(0)=0$ (0.5)

1. 2^{n-1}
2. $2n$
3. n
4. $2(n-1)$

Q4. Whats the best-case time complexity of Bubble Sort algorithm with early termination? (0.5)

5. $\Theta(1)$
6. $\Theta(\log n)$
7. $\Theta(n)$
8. $\Theta(n \log n)$

Q5. Raj wants to do a round trip (ignoring the directions) between the cities Manipal, Udupi, Shimoga, Bangalore, Mangalore, and Davangere, starting from Manipal and covering all the cities. He uses Travelling Salesman Problem (TSP) using Brute Force approach to make sure that he doesn't pass through any of the roads multiple times. Provided these conditions, how many round trips are possible? (0.5)

1. 6
2. 120
3. 720
4. 36

Q6. When Brute string matching algorithm is used on a pattern of length 10 and text of length 100, what would be best case and worst case number of character comparisons respectively? (0.5)

1. 100, 1000
2. 10, 910
3. 50, 500
4. 50, 910

Q7. List all the keys of the given array that will require the largest number of key comparisons when searched for by using the binary search, {3, 14, 27, 31, 39, 42, 55, 70, 74, 81, 85} (0.5)

1. 3 and 85
2. 3 and 42
3. 55 and 85
4. 42 and 55

Q8. What is the time efficiency of $T(n) = 8T\left(\frac{n}{2}\right) + 1000n^2$ after applying the Master's method. (0.5)

1. $\Theta(n)$
2. $\Theta(n^3)$
3. $\Theta(n^2)$
4. $\Theta(n^4)$

Q9. Strassen's algorithm requires how many scalar multiplications to multiply two 8×8 matrices. (0.5)

1. **343
2. 512
3. 256
4. 216

Q10. Large integer multiplication using divide & conquer method for multiplying two 16-digit numbers requires how many single-digit multiplications. (0.5)

1. 256
2. **81
3. 64
4. 27

Type: DES

Q11. The key aspect of algorithm analysis involves the identification of the most important operation of the algorithm, called the basic operation which contributes the most to the total running time. Consider that the mechanism of dot(.) product is adopted to find the product $C=A*B$ of two $N \times N$ matrices, A and B . Write an algorithm to find the multiplication of two matrices. Analyze its time efficiency in terms of the number of times the algorithm's basic operation is performed. (3)

Ans:

Scheme:

Writing the algorithm with all its components – 1.5 M

Description of time efficiency and its analysis – 1.5 M

Solution:

```
ALGORITHM MatrixMultiplication( $A[0..n-1, 0..n-1]$ ,  $B[0..n-1, 0..n-1]$ )
//Multiplies two square matrices of order  $n$  by the definition-based algorithm
//Input: Two  $n \times n$  matrices  $A$  and  $B$ 
//Output: Matrix  $C = AB$ 
for  $i \leftarrow 0$  to  $n-1$  do
    for  $j \leftarrow 0$  to  $n-1$  do
         $C[i, j] \leftarrow 0.0$ 
        for  $k \leftarrow 0$  to  $n-1$  do
             $C[i, j] \leftarrow C[i, j] + A[i, k] * B[k, j]$ 
return  $C$ 
```

We measure an input's size by matrix order n . There are two arithmetical operations in the innermost loop here—multiplication and addition—that, in principle, can compete for designation as the algorithm's basic operation. Actually, we do not have to choose between them, because on each repetition of the innermost loop each of the two is executed exactly once. So by counting one we automatically count the other.

We consider multiplication as the basic operation. Let us set up a sum for the **total number of multiplications $M(n)$** executed by the algorithm. There is just **one multiplication executed on each repetition of the algorithm's innermost loop**, which is governed by the

variable **k** ranging from the **lower bound 0** to the **upper bound n-1**. Therefore, the number of multiplications made for every pair of specific values of variables i and j is,

$$\sum_{k=0}^{n-1} 1,$$

and the total number of multiplications $M(n)$ is expressed by the following triple sum:

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1.$$

Now, we can compute this sum by using formula (S1) and rule (R1) given above. Starting with the innermost sum $\sum_{k=0}^{n-1} 1$, which is equal to n , we get

$$M(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} \sum_{k=0}^{n-1} 1 = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} n = \sum_{i=0}^{n-1} n^2 = n^3.$$

$$\sum_{i=l}^u 1 = u - l + 1 \quad \text{where } l \leq u \text{ are some lower and upper integer limits, (S1)}$$

$$\sum_{i=l}^u ca_i = c \sum_{i=l}^u a_i, \quad \text{(R1)}$$

Q12. Sort the characters of the string “**SELECT**” in ascending order using Selection Sort algorithm by clearly showing all the steps. Compute the number of key comparisons and the number of necessary swaps required to obtain the sorted array. **(3)**

ANS:

- Array = [S, E, L, E, C, T], n=6
- $i = 0$ (Pass 1):-
 - min = 0
 - j = 1:- min = 1
 - j = 2:- min = 2
 - j = 3:- min = 3
 - j = 4:- min = 4
 - j = 5:- min = 4
 - Swap(A[0], A[4])
 - Array = [C, E, L, E, S, T]
- $i = 1$ (Pass 2):-

- $\text{min} = 1$
- $j = 2$:- $\text{min} = 1$
- $j = 3$:- $\text{min} = 1$
- $j = 4$:- $\text{min} = 1$
- $j = 5$:- $\text{min} = 1$
- $\text{Swap}(A[1], A[1])$
- $\text{Array} = [C, E, L, E, S, T]$

○ $i = 2$ (*Pass 3*):-

- $\text{min} = 2$
- $j = 3$:- $\text{min} = 3$
- $j = 4$:- $\text{min} = 3$
- $j = 5$:- $\text{min} = 3$
- $\text{Swap}(A[2], A[3])$
- $\text{Array} = [C, E, E, L, S, T]$

○ $i = 3$ (*Pass 4*):-

- $\text{min} = 3$
- $j = 4$:- $\text{min} = 3$
- $j = 5$:- $\text{min} = 3$
- $\text{Swap}(A[3], A[3])$
- $\text{Array} = [C, E, E, L, S, T]$

○ $i = 4$ (*Pass 5*):-

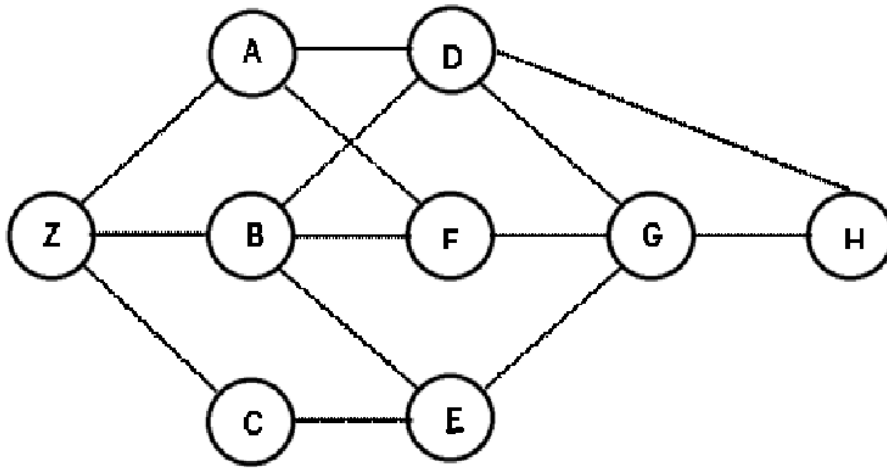
- $\text{min} = 4$
- $j = 5$:- $\text{min} = 4$
- $\text{Swap}(A[4], A[4])$
- $\text{Array} = [C, E, E, L, S, T]$

○ Number of Comparisons = $5 + 4 + 3 + 2 + 1 = 15$

○ Necessary Number of Swaps (in Pass 1 and Pass 3) = 2

(0.5 Marks for steps for each pass, 0.5 marks for total number of comparisons and necessary number of swaps)

Q13. Traverse the graph shown in Figure below by breadth first search. Write the traversal order and construct the corresponding breadth first search tree. Start the traversal at vertex A and resolve ties by the vertex alphabetical order. Also list the cross edges encountered.



(4)

ANS:

Q No.

QUEUE

A	
---	--

A	D	F	Z
--------------	---	---	---

A

A	D	F	Z	B	G	H
--------------	--------------	---	---	---	---	---

A, D

A	D	F	Z	B	G	H
--------------	--------------	--------------	---	---	---	---

A, D, F

A	D	F	Z	B	G	H	C
--------------	--------------	--------------	---	---	---	---	---

A, D, F, Z

A	D	F	Z	B	G	H	C	E
--------------	--------------	--------------	---	---	---	---	---	---

A, D, F, Z, B

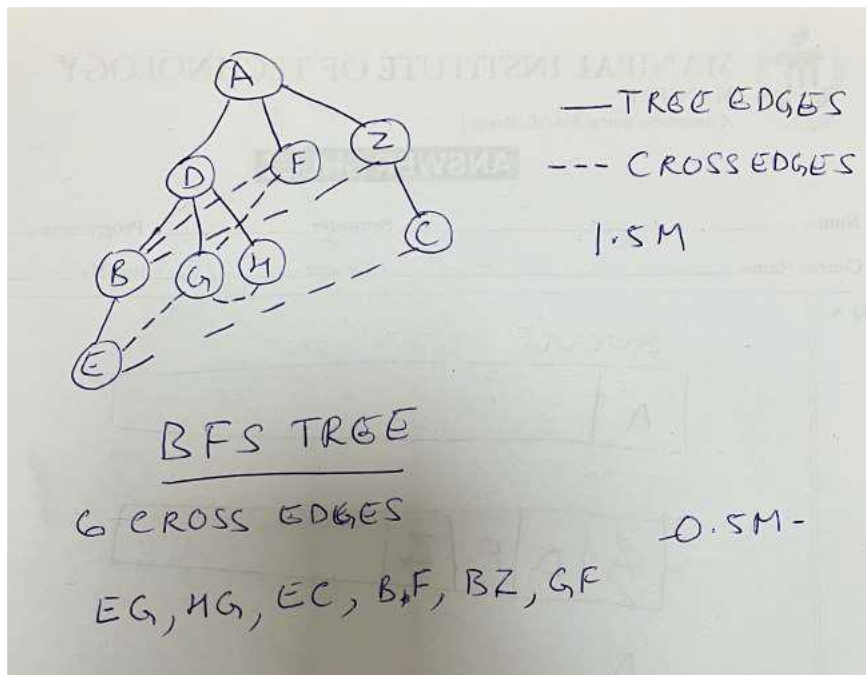
A, D, F, Z, B, G

A, D, F, Z, B, G, H

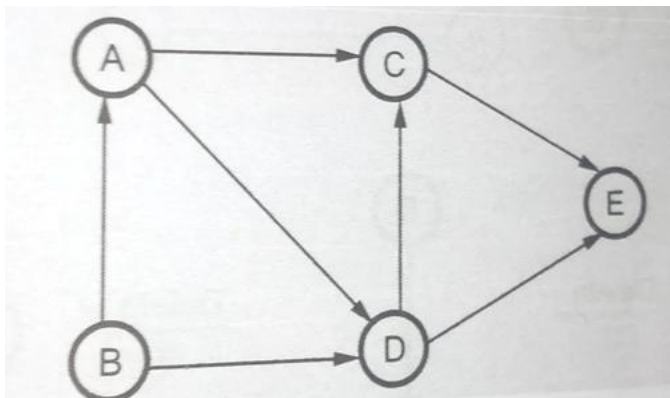
A, D, F, Z, B, G, H, C

2M-

BFS
TRAVERSAL ORDER - A₁, D₂, F₃, Z₄, B₅, G₆, H₇, C₈, E₉



Q14. How to find the topological sorting of a given Directed Acyclic Graph(DAG) using source removal method? Apply this method to find the topological sorting of the following DAG



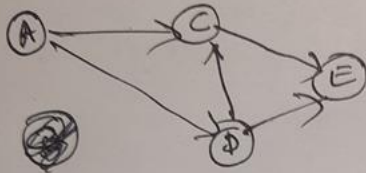
(3)

ANS:

Repeatedly identify and remove a source (a vertex with no incoming edges) and all the edges incident to it until either no vertex is left or there is no source among the remaining vertices (not a dag)

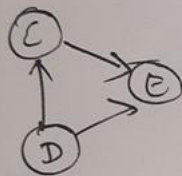
1Mark

Indegree of vertex B is 0, so remove it & all its incident edges. So remaining graph



0.5 M

Now Indegree of A is zero So remove it



0.5 M

④ D indegree is zero, so remove it



0.5 m

C in degree '0', remove it



Remove F

0.5 M

So Topological sorting is B, A, D, C, E

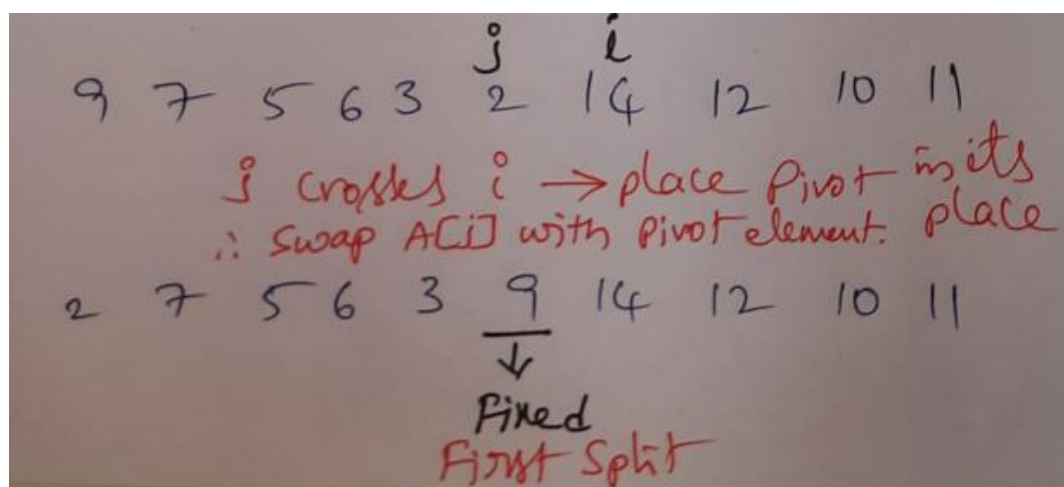
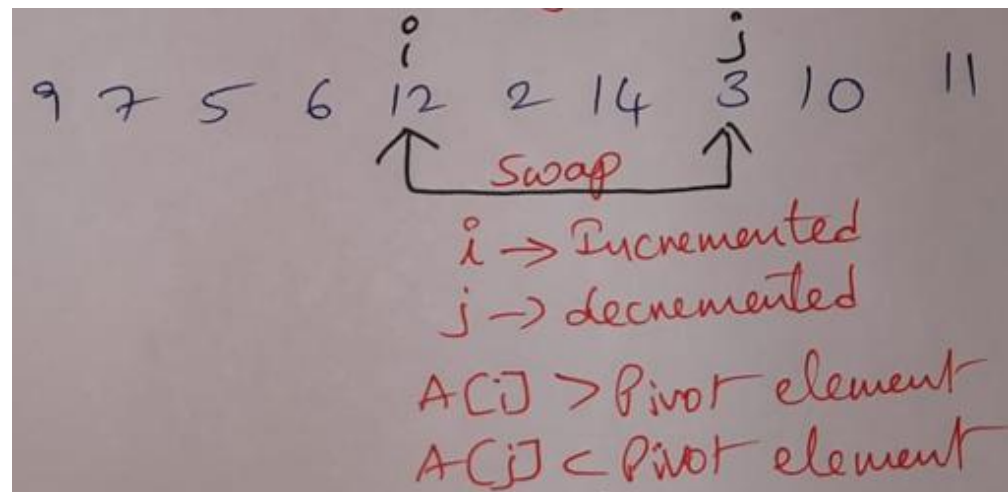
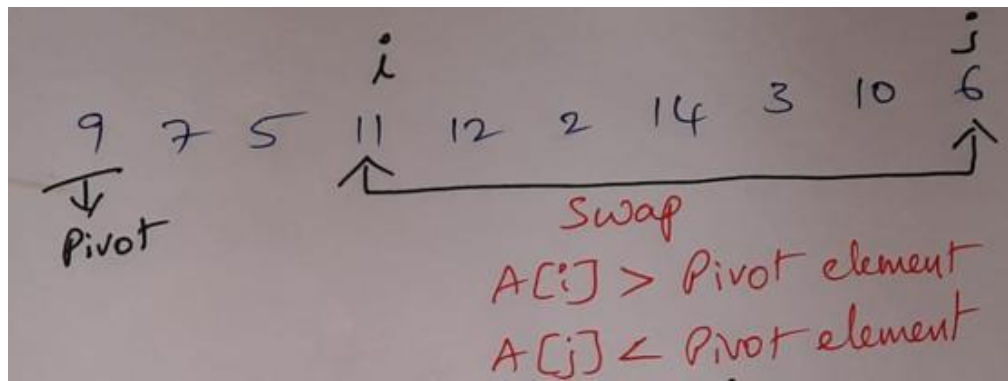
Q15. Apply quicksort on the list 9, 7, 5, 11, 12, 2, 14, 3, 10, 6 to sort in ascending order using the Hoare partition by considering the first element as a pivot. Construct a tree of recursive calls for the same (4)

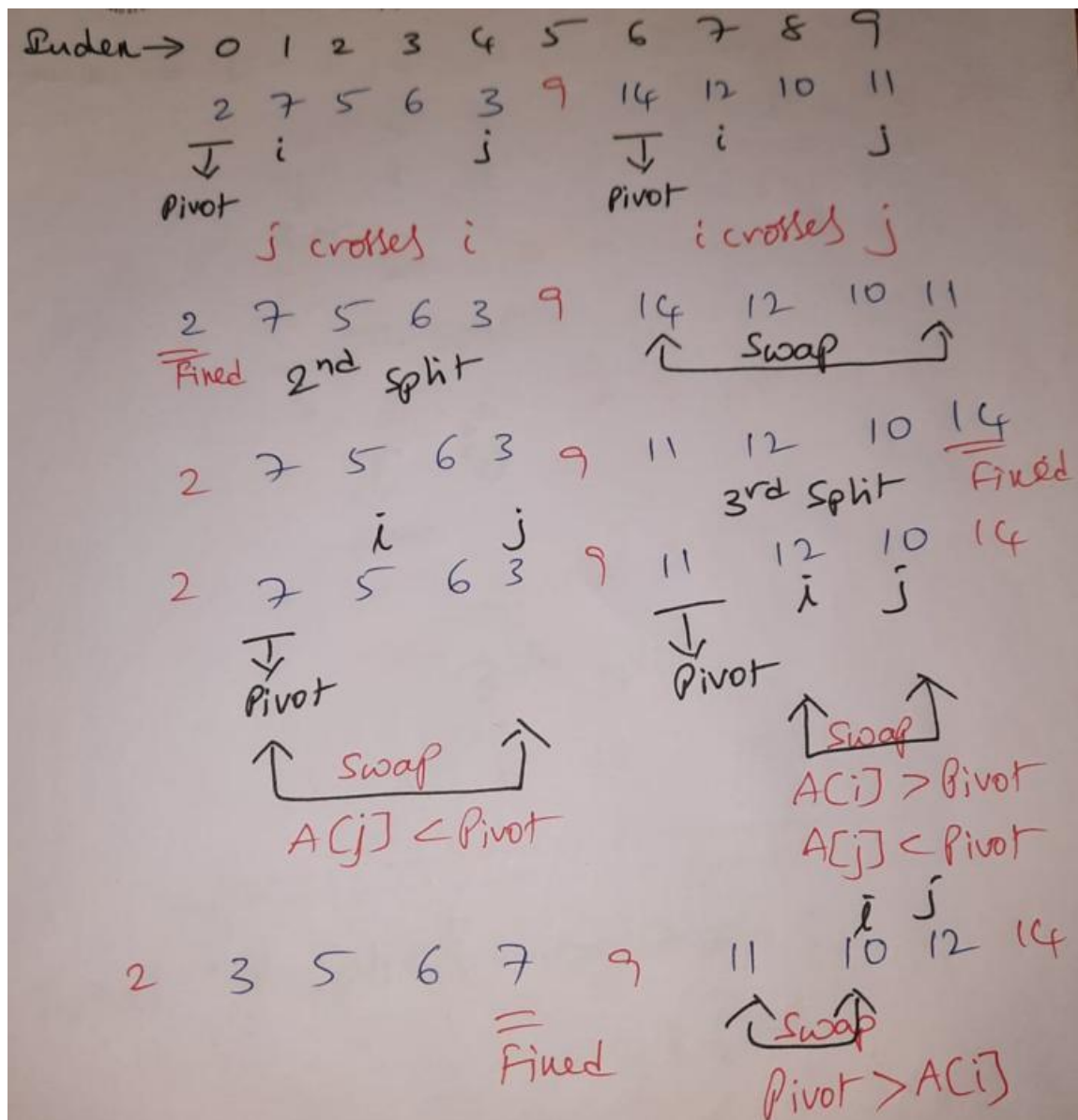
ANS:

Quicksort of the given list 9, 7, 5, 11, 12, 2, 14, 3, 10, 16 and the pivot is 9 (first element)

Given List	Index 0	1	2	3	4	5	6	7	8	9
	9	7	5	11	12	2	14	3	10	6

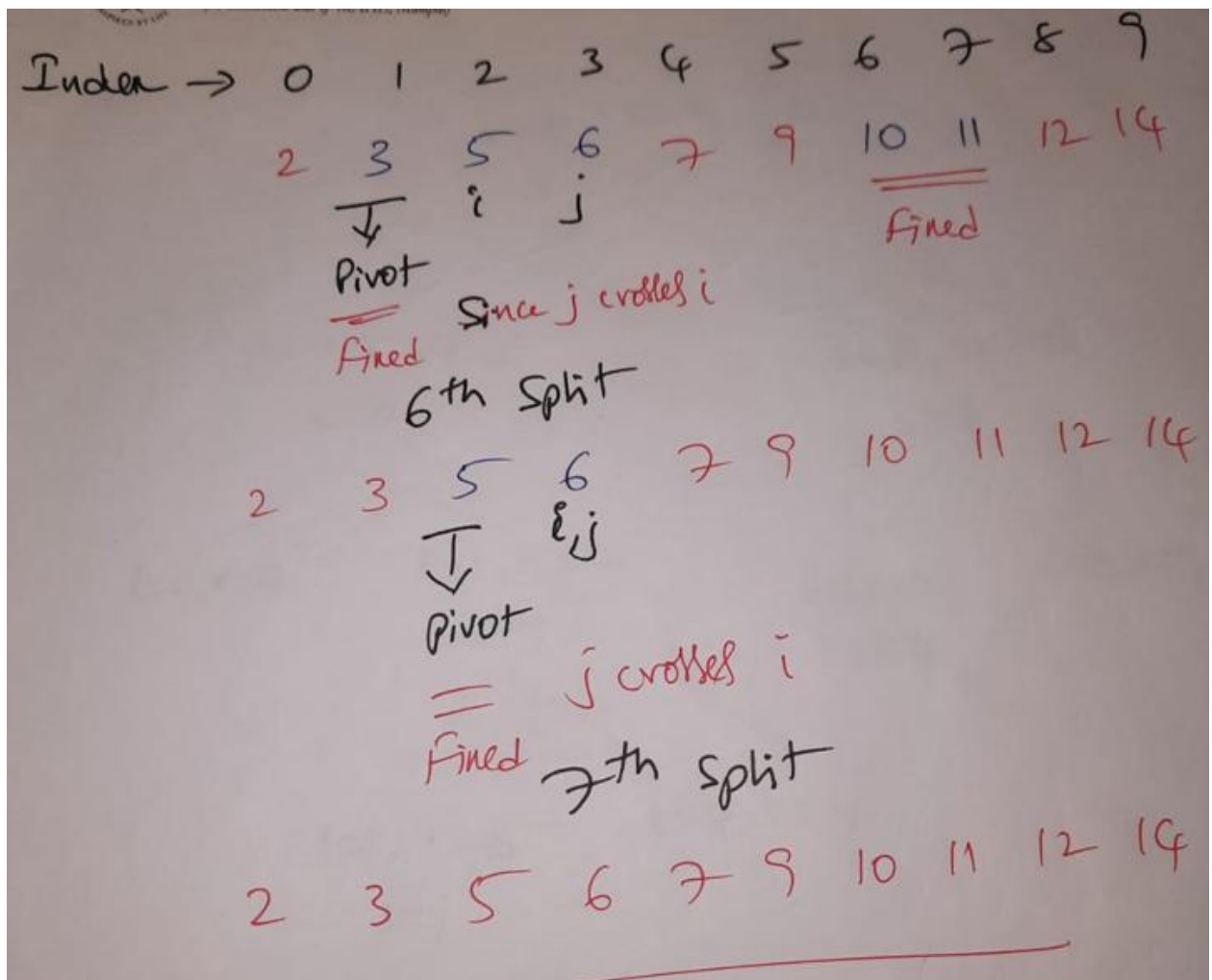
Pivot $\rightarrow$ 9 (First element)





2 3 5 6 7 9 10 11 12 14

4th split Fined 5th split Fined

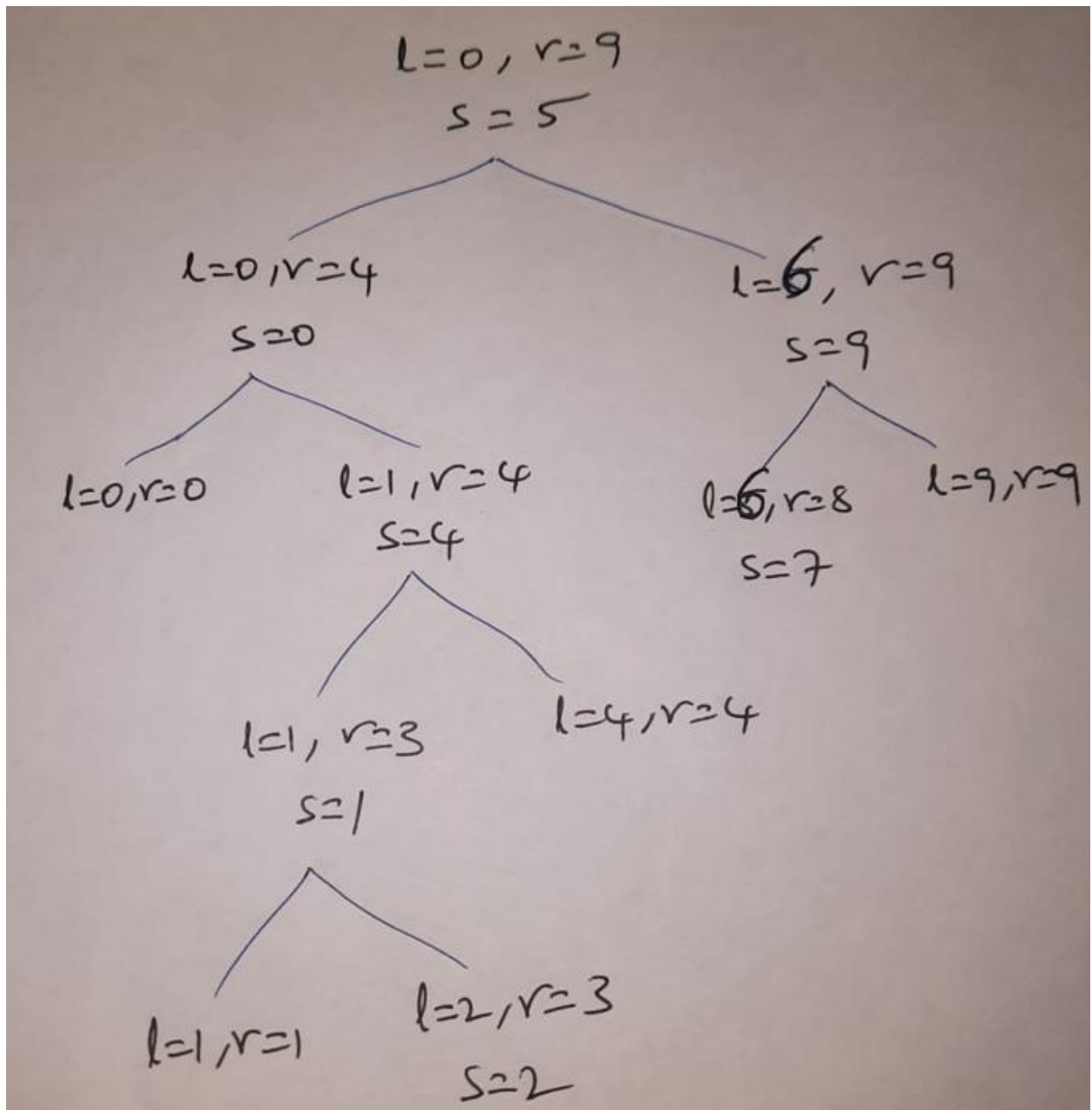


7 splits, so $7 \times 0.5 = 3.5$ Marks

0.5 Marks for the tree

Total = $3.5 + 0.5 = 4$ M

Recursion Tree

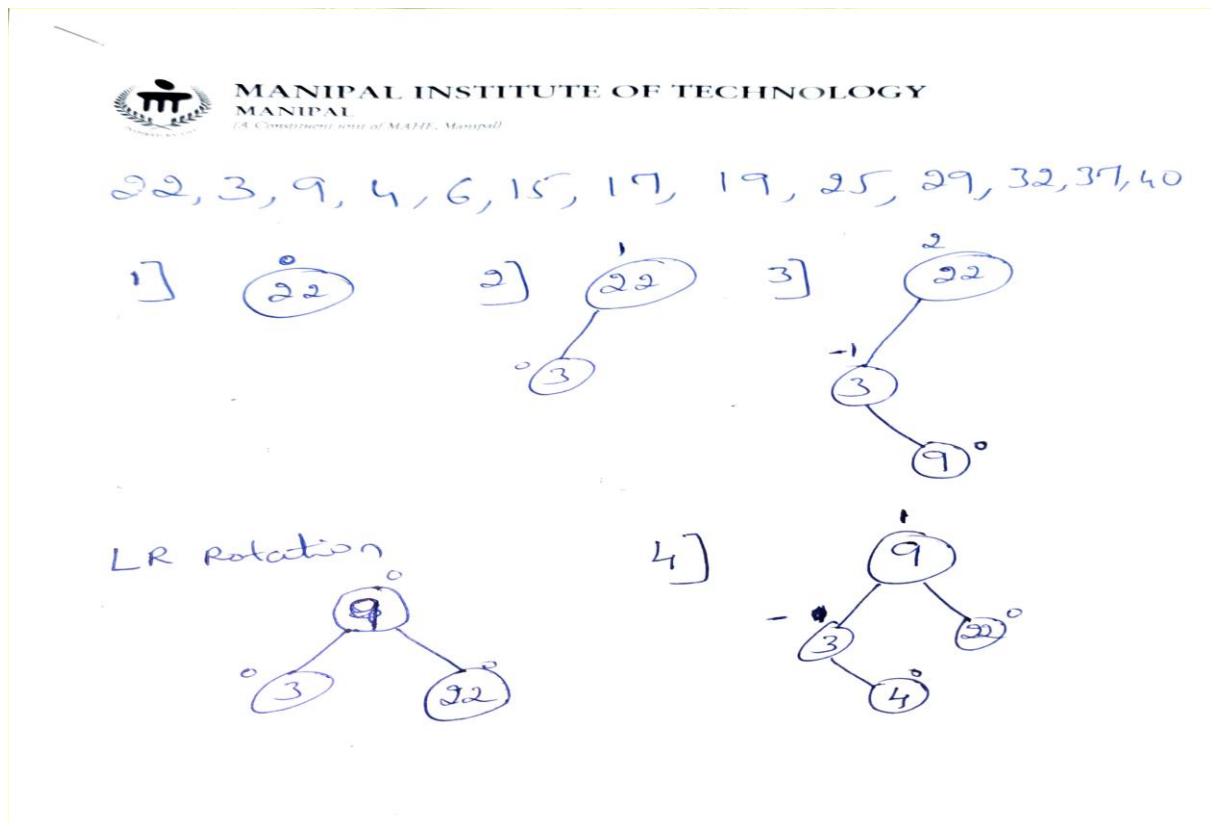


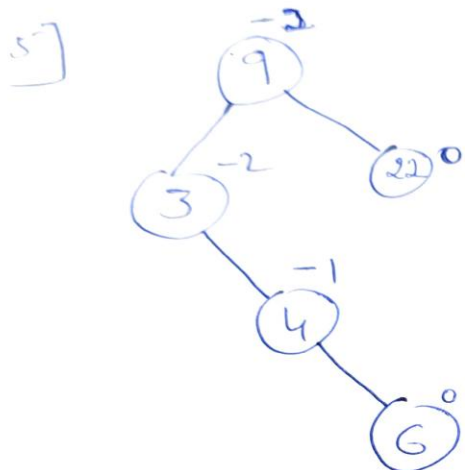
Q16. Discuss AVL trees along with its properties and drawbacks. Construct AVL tree by showing the successive insertion of key elements and rotations involved in each step for the given list of elements 22, 3, 9, 4, 6, 15, 17, 19, 25, 29, 32, 37, 40. (4)

ANS:

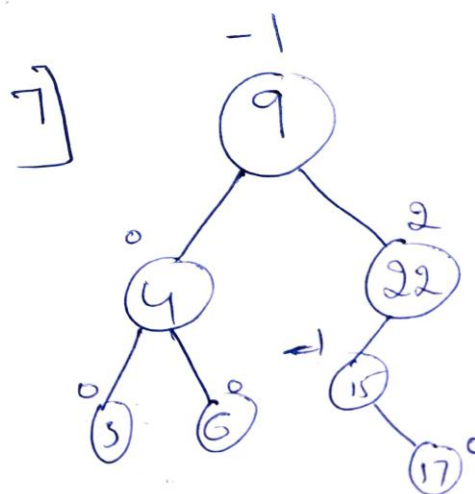
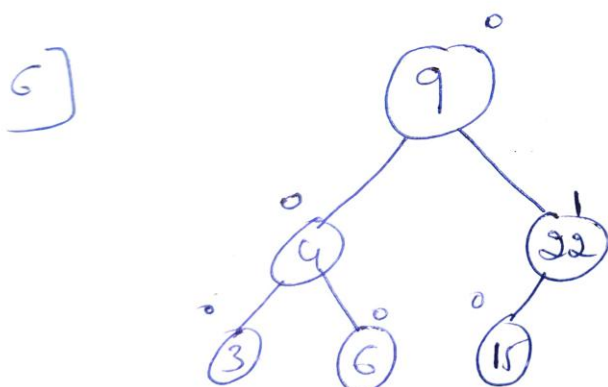
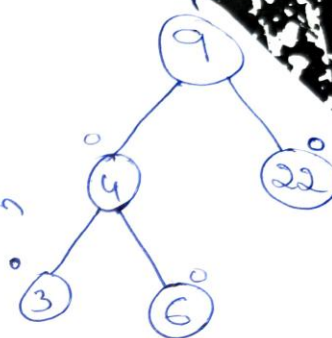
an unbalanced binary search tree is transformed into a balanced one. Because of this, such trees are called **self-balancing**. An **AVL tree** requires the difference between the heights of the left and right subtrees of every node never exceed 1. If an insertion or deletion of a new node creates a tree with

a violated balance requirement, the tree is restructured by one of a family of special transformations called **rotations** that restore the balance required. The drawbacks of AVL trees are frequent rotations and the need to maintain balances for its nodes. These drawbacks have prevented AVL trees from becoming the standard structure for implementing dictionaries.

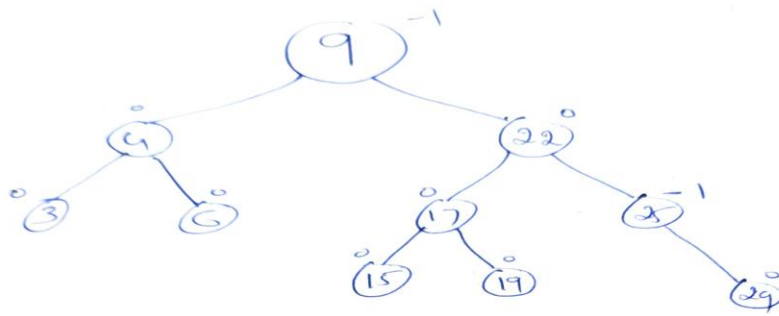




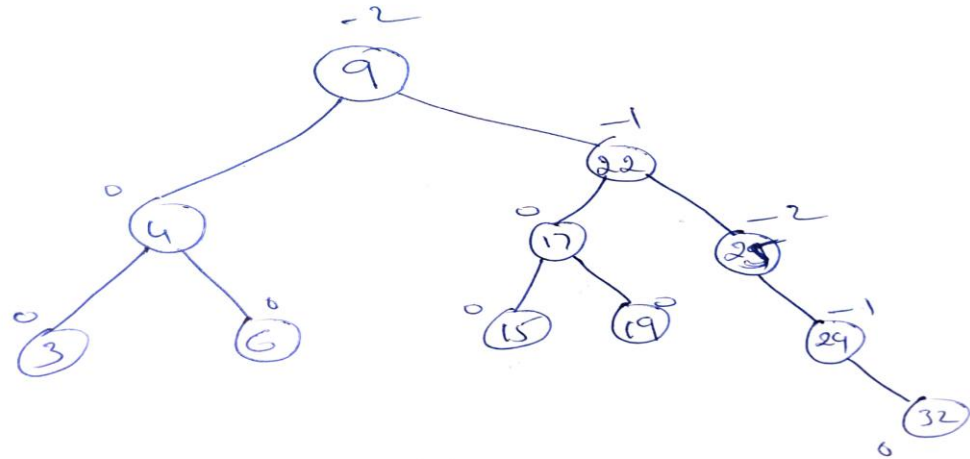
Right
Rotation



LR Rotation



11]

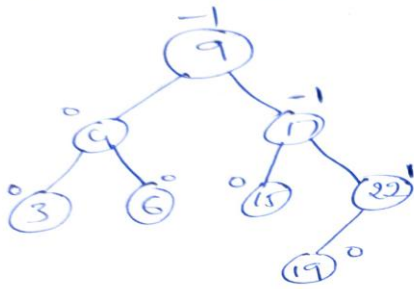


Left rotation

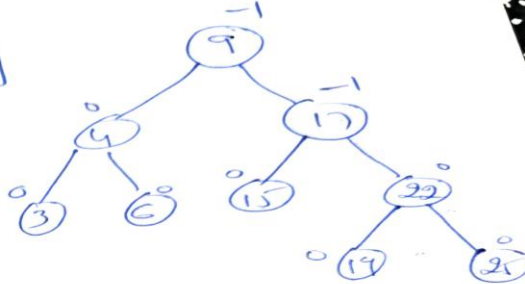


MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A Constituent Inst. of MAITE, Manipal)

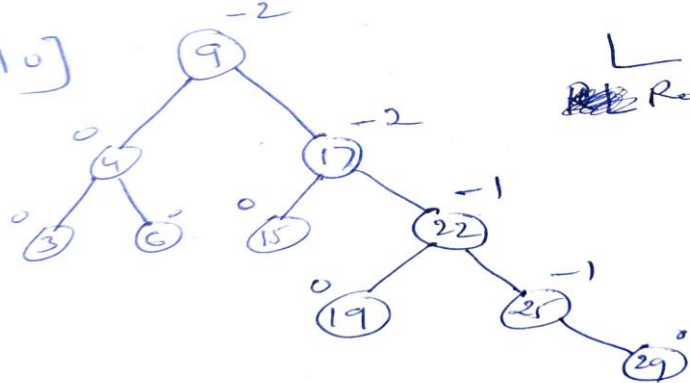
8]



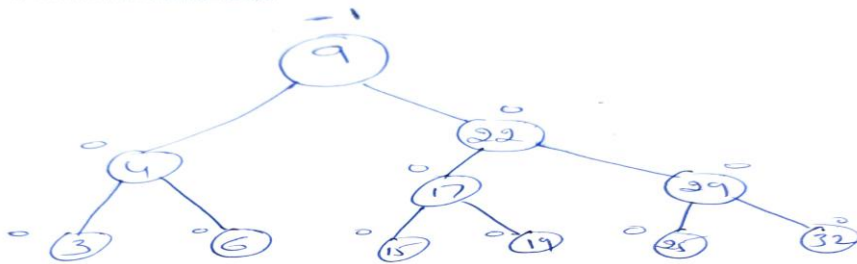
9]



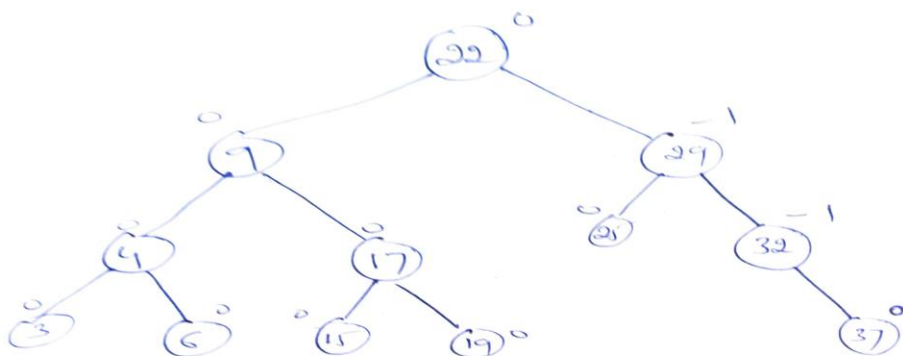
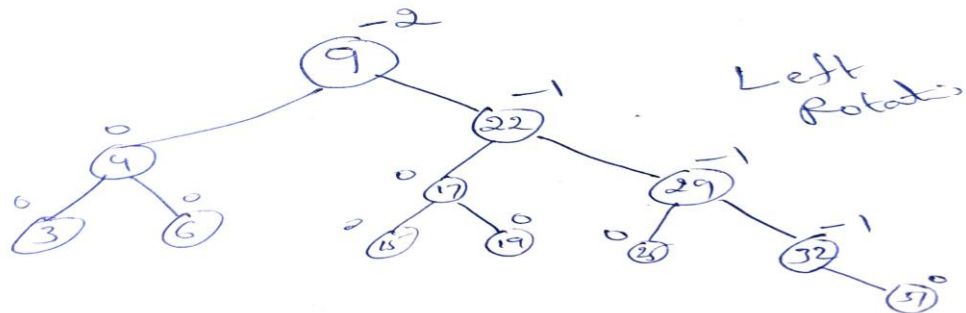
10]



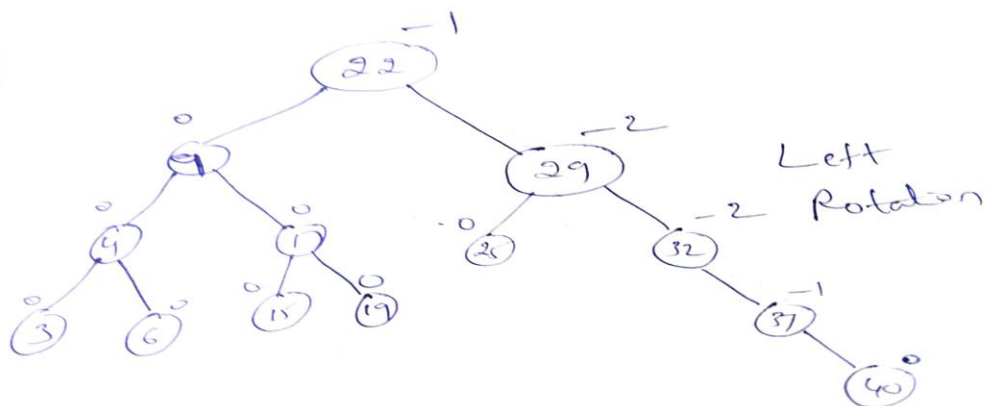
Left Rotation

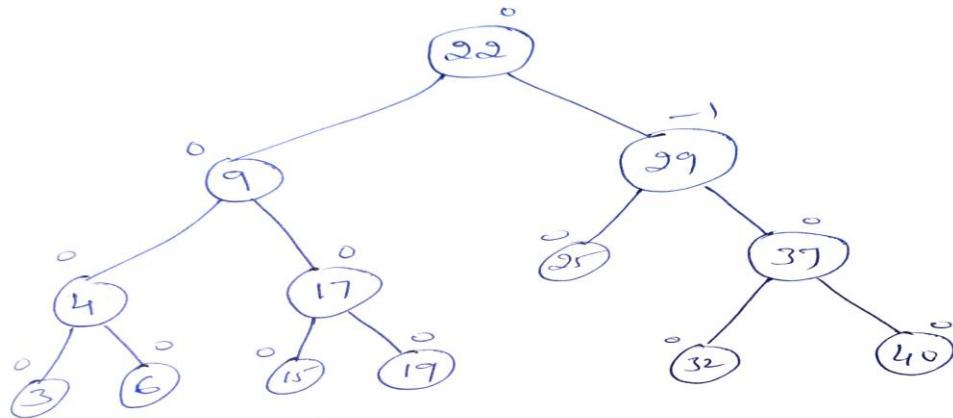


12]



13]





Final Answer.

AVL Tree definition & draw back — 0.5M
Each rotation carries 0.5M

Total — $0.5 + 0.5 \times 7 = 4M$

Q17. Sort the list 82, 90, 10, 12, 15, 77, 55, 23 in ascending order using Heap Sort. Show all trees in detail during the bottom-up heap construction and sorting (Tree representation). **(4)**

ANS:

Step 1 - Construct a Heap with given list of unsorted numbers and convert to Max Heap



list of numbers after heap converted to Max Heap

90, 82, 77, 23, 15, 10, 55, 12

Step 2 - Delete root (90) from the Max Heap. To delete root node it needs to be swapped with last node (12). After delete tree needs to be heapify to make it Max Heap.



list of numbers after swapping 90 with 12.

12, 82, 77, 23, 15, 10, 55, 90

Step 3 - Delete root (82) from the Max Heap. To delete root node it needs to be swapped with last node (55). After delete tree needs to be heapify to make it Max Heap.



list of numbers after swapping 82 with 55.

12, 55, 77, 23, 15, 10, 82, 90

Step 4 - Delete root (77) from the Max Heap. To delete root node it needs to be swapped with last node (10). After delete tree needs to be heapify to make it Max Heap.



list of numbers after swapping 77 with 10.

12, 55, 10, 23, 15, 77, 82, 90

Step 5 - Delete root (55) from the Max Heap. To delete root node it needs to be swapped with last node (15). After delete tree needs to be heapify to make it Max Heap.



list of numbers after swapping 55 with 15.

12, 15, 10, 23, 55, 77, 82, 90

Step 6 - Delete root (23) from the Max Heap. To delete root node it needs to be swapped with last node (12). After delete tree needs to be heapify to make it Max Heap.



list of numbers after swapping 23 with 12.

12, 15, 10, 23, 55, 77, 82, 90

Step 7 - Delete root (15) from the Max Heap. To delete root node it needs to be swapped with last node (10). After delete tree needs to be heapify to make it Max Heap.



list of numbers after Deleting 15, 12 & 10 from the Max Heap.

10, 12, 15, 23, 55, 77, 82, 90

Whenever Max Heap becomes Empty, the list get sorted in Ascending order